

Business Requirements Document (BRD)

Project: Customer & Order Data Ingestion, SCD Type 2 Orders Summary, and Customer Aggregate Spend

Prepared by: Senior Business Analyst

Date: 2026-03-29

Version: 1.0

1. Executive Summary

- Purpose: Define business and functional requirements to read customer and order CSV data, cleanse and transform it, persist as Delta tables, create an SCD Type 2 orders summary, and produce daily customer aggregate spend. The solution will be implemented using Databricks Delta Lake and Delta Live Tables (DLT).
- Goals:
- Reliable ingestion of customer and order CSV sources into Delta format.
- Standardized schemas for customer, order, ordersummary, and customeraggregatespend tables.
- Data quality rules: remove null and "Null" values and deduplicate.
- Calculate TotalAmount for orders.
- Maintain ordersummary as SCD Type 2 to preserve customer history.
- Produce daily aggregated spend per customer (Name, Date, TotalAmount).
- Automate the pipeline using Delta Live Tables.

2. Stakeholders

- Business Owner: Sales/Revenue Analytics team
- Data Owner: Data Engineering team (Databricks / Lakehouse)
- Consumers: BI/Reporting, Finance, Marketing, Customer Success
- Compliance & Security: Data Governance, Security teams
- Implementation Team: Data Engineers, ETL Developers, QA Engineers

3. Scope

3.1 Included

- Read CSV files from specified volumes/paths for customer and order datasets.
- Create/overwrite Delta tables for customer and order.
- Apply cleansing rules (null / "Null" removal, deduplication).
- Add TotalAmount to order records.
- Create ordersummary table in catalog gen_ai_poc_databrickscoe.sdmc_wizard if not exists.
- Join customer and order on CustId and load into ordersummary with SCD Type 2 semantics.

- Implement logic to update SCD Type 2 ordersummary when customer attributes change.
- Create customeraggregatespend table and populate it from aggregated ordersummary.
- Implement all processing using Delta Live Tables for automated orchestration, lineage, and monitoring.

3.2 Excluded

- Ingestion from sources other than the provided CSV file paths.
- Upstream changes to CSV schema outside specified fields (unless change control raises a requirement).
- Real-time streaming ingestion (current requirement is file-based batch via DLT).
- Downstream visualization and dashboard creation (outside scope).

4. Assumptions

- The CSV files for customer and order reside at:
- /Volumes/gen_ai_poc_databrickscoe/sdlc_wizard/customerdata
- /Volumes/gen_ai_poc_databrickscoe/sdlc_wizard/orderdata
- Files are readable by Databricks workspace and appropriate IAM and storage mounts are configured.
- The provided schemas are the authoritative fields for ingestion. Any additional columns will be handled via change control.
- Timestamps/timezone handling for Date fields will be standardized (see Nonfunctional).
- Name is a sufficiently stable identifier for aggregation (customer name may not be unique; business to confirm if aggregation should use CustId instead).
- Delta Live Tables runtime and required cluster resources are available.

5. Requirements

5.1 Functional Requirements (FR)

FR-1: Ingest source CSV data

- FR-1.1: Read customer CSV files from /Volumes/gen_ai_poc_databrickscoe/sdlc_wizard/customerdata and write to Delta table "customer".
- FR-1.2: Read order CSV files from /Volumes/gen_ai_poc_databrickscoe/sdlc_wizard/orderdata and write to Delta table "order".

FR-2: Source schemas

- FR-2.1: customer schema must include fields: CustId, Name, EmailId, Region.
- FR-2.2: order schema must include fields: OrderId, ItemName, PricePerUnit, Qty, Date, CustId.

FR-3: Data enrichment

- FR-3.1: Compute TotalAmount on order as PricePerUnit * Qty and add as new column TotalAmount.

FR-4: Data quality cleansing

- FR-4.1: Remove records where any required field is null or the literal string "Null".
- FR-4.2: Remove duplicate records in both customer and order tables. Deduplication strategies must be defined per table (see Design & Rules).

FR-5: Orders summary table

- FR-5.1: Create table ordersummary in catalog gen_ai_poc_databrickscoe, schema sdhc_wizard if it does not exist.
- FR-5.2: ordersummary must contain fields: CustId, Name, EmailId, Region, OrderId, ItemName, PricePerUnit, Qty, Date, TotalAmount (TotalAmount is implied from step 3).

FR-6: SCD Type 2 behavior for ordersummary

- FR-6.1: ordersummary must be maintained as SCD Type 2.
- FR-6.2: When a customer record changes (e.g., Name, EmailId, Region), existing records related to that customer should be marked Inactive and assigned an EndDate.
- FR-6.3: Insert new records for the new customer state as Active with StartDate.
- FR-6.4: Maintain history for all changes; do not delete historical rows.

FR-7: Customer aggregate spend

- FR-7.1: Create table customeraggregatespend in catalog gen_ai_poc_databrickscoe, schema sdhc_wizard if not exists, with columns: Name, TotalAmount, Date.
- FR-7.2: Populate customeraggregatespend by aggregating ordersummary: GROUP BY Name, Date with SUM(TotalAmount) as TotalAmount.

FR-8: Implementation orchestration

- FR-8.1: Implement steps FR-1 through FR-7 using Delta Live Tables (DLT) pipelines.
- FR-8.2: Pipeline should support incremental processing and be idempotent.

5.2 Non-Functional Requirements (NFR)

NFR-1: Performance

- NFR-1.1: Pipeline must complete daily batch within a defined SLA (to be agreed — e.g., within 1 hour of data availability).
- NFR-1.2: Aggregations should be optimized using partition pruning (e.g., partition by Date) and suitable file sizes per Delta best practices.

NFR-2: Reliability and Monitoring

- NFR-2.1: DLT must provide visibility, metrics, and alerts on pipeline failures, data quality rule violations, and throughput.
- NFR-2.2: Implementation must support retries and resumability.

NFR-3: Data Retention & History

- NFR-3.1: SCD Type 2 ordersummary must preserve full history with StartDate and EndDate fields for each record; default Active flag indicates current records.
- NFR-3.2: Retention policy for historical records should be determined by data governance (e.g., keep 7 years) and implemented as a separate archival process.

NFR-4: Security & Access

- NFR-4.1: Only authorized roles can read/write the target catalog gen_ai_poc_databrickscoe.sdlc_wizard.
- NFR-4.2: Sensitive fields (e.g., EmailId) must follow data privacy rules; encryption at rest and in transit required.
- NFR-4.3: Audit logging for changes to SCD table required.

NFR-5: Data Format & Type Standards

- NFR-5.1: Standardize Date to ISO-8601 (YYYY-MM-DD) or timestamp with timezone; specify timezone handling.
- NFR-5.2: PricePerUnit, Qty, TotalAmount should be stored as appropriate numeric types (e.g., DECIMAL(18,4) for currency).
- NFR-5.3: CustId and OrderId types defined consistently (string or numeric) — choose string to avoid leading-zero issues unless business confirms numeric.

6. Detailed Design & Business Rules

6.1 Source ingestion

- Use DLT to create two streaming or batch tables (depending on file arrival): Bronze stage Delta tables "customer_bronze" and "order_bronze" from CSV source paths. Include ingestion metadata columns (e.g., _ingest_time, _file_name) for traceability.

6.2 Schema enforcement & parsing

- Enforce expected column list; if extra columns exist, log and drop them unless change request is approved.
- Implement parsing rules: trim strings, normalize casing for Region, and validate numeric and date formats.

6.3 Data quality rules

- Null handling:
- Define required fields:

- customer: CustId (required), Name (required), EmailId (optional/required per business), Region (optional).
- order: OrderId (required), PricePerUnit (required), Qty (required), Date (required), CustId (required).
- Remove rows where any required field is null, empty string, or the literal "Null" (case-insensitive).
- Log rejected rows to a quarantine/error Delta table with reason codes.
- Deduplication:
 - customer dedupe key: CustId (primary); if multiple records for same CustId in same batch, choose most recent by ingest timestamp or last-modified column; log duplicates.
 - order dedupe key: OrderId; if duplicates, keep single authoritative record (e.g., latest ingest), log duplicates.

6.4 Calculations

- TotalAmount = PricePerUnit * Qty; compute with numeric precision and round per currency rules.
- Ensure null-safe multiplication (treat null PricePerUnit or Qty as reject per data quality).

6.5 Ordersummary SCD Type 2 design

- Ordersummary table schema (suggested final physical schema):
 - CustId (PK candidate)
 - Name
 - EmailId
 - Region
 - OrderId
 - ItemName
 - PricePerUnit
 - Qty
 - Date (order date)
 - TotalAmount
 - StartDate (timestamp) — when this record became active
 - EndDate (timestamp) — when this record ceased to be active; NULL for active
 - IsActive (boolean) — current record indicator
 - RecordHash or ChangeHash (optional) — to detect changes efficiently
 - SourceMetadata (optional) — e.g., _file_name, _ingest_time
- SCD Type 2 logic:
 - When new orders arrive joined with current customer state:
 - If the combination of business key (CustId + OrderId) does not exist, insert a new record with StartDate = processing date/time, EndDate = NULL, IsActive = true.

- If the combination exists but customer attributes (Name, EmailId, Region) differ from the current active record (detected via RecordHash or field comparison), then:
- Update existing active record: set EndDate = processing timestamp, IsActive = false.
- Insert new record with updated customer attributes and StartDate = processing timestamp, EndDate = NULL, IsActive = true.
- If only order-level details changed (Price, Qty, ItemName), evaluate whether to treat as new version; by default, preserve history for customer attribute changes primarily. Business to confirm whether order edits create new SCD rows.
- Implement SCD updates using Delta MERGE or DLT-managed expectation/streams to ensure idempotency.

6.6 Aggregation to customeraggregatespend

- Aggregation logic:
- Source: ordersummary effective rows (including only active or all rows as per business rule — default use Date field in ordersummary).
- Group by Name and Date (Date should be truncated to date granularity).
- SUM(TotalAmount) as TotalAmount.
- Target table customeraggregatespend schema:
- Name
- Date
- TotalAmount
- Partition and indexing:
- Partition by Date for performance.
- If Name cardinality high, consider clustering instead of partitioning by Name.

6.7 Delta Live Tables implementation

- Build DLT pipeline with following tables/views:
- customer_bronze (raw ingest)
- order_bronze (raw ingest)
- customer_clean (quality-checked, deduped)
- order_clean (quality-checked, deduped, TotalAmount added)
- ordersummary_scd (SCD Type 2 physical table)
- customeraggregatespend (materialized aggregation)
- Use DLT expectations for data quality rules (null checks, uniqueness) and route violations to quarantine.
- Leverage DLT incremental model: use streaming/file-based auto-ingest to handle new files.
- Use DLT's plan or merge transformations to manage SCD Type 2 merges.

7. Data Validation & Test Cases

- Ingestion tests:
- Provide sample CSVs with normal and anomalous rows. Confirm that valid rows are loaded to delta and invalid rows to quarantine.
- TotalAmount tests:
- Cases with integer and decimal PricePerUnit and Qty; validate computed TotalAmount and rounding.
- Null & "Null" tests:
- Rows containing literal "Null" in different cases and whitespace; confirm they are rejected if in required fields.
- Deduplication tests:
- Duplicate customer rows in same batch; verify retention of single record and logging.
- Duplicate order rows; verify OrderId uniqueness enforcement.
- SCD Type 2 tests:
- Update a customer's Name or Region and ingest; confirm previous row EndDate is set and new active row created.
- Insert order for existing and new customers; validate SCD behavior.
- Aggregation tests:
- Multiple orders per Name and Date; confirm SUM(TotalAmount) accuracy.

8. Error Handling & Logging

- Quarantine tables for customer_rejected and order_rejected with fields: raw_row, reason_code, reason_message, ingest_metadata.
- Operational logging for DLT: pipeline start/stop, failures, table-level metrics (rows read, rows written, expectations failed).
- Alerts: Notify Data Engineering and Business Owners on pipeline failures and high rejection rates.

9. Security, Compliance, and Governance

- Access control using Unity Catalog (or equivalent) to restrict operations on gen_ai_poc_databrickscoe.sdlc_wizard.*.
- Encryption at rest and in transit as per organizational policy.
- PII handling: Flag EmailId as sensitive; mask or limit access in downstream datasets unless authorized.
- Data lineage: DLT provides lineage; ensure it's captured for audits.

10. Deployment & Operational Considerations

- Environments: dev, test, staging, prod with separate catalogs/schemas or prefixes.
- CI/CD: Store DLT notebook or pipeline definitions in Git; CI/CD to deploy pipeline changes across environments.

- Schedule: DLT pipeline trigger schedule (e.g., every hour or daily); configurable.
- Resource sizing: Determine cluster sizes and autoscaling policies to meet SLA.
- Backups & Recovery: Delta table backups/versioning and time travel retention policies configured.

11. Open Decisions & Questions

- Q1: Should customeraggregatespend group by CustId instead of Name to avoid ambiguity with duplicate names? Recommendation: Use CustId + Name and preserve display Name for reports; business to confirm.
- Q2: Which fields constitute a customer change that triggers SCD Type 2? (Name, EmailId, Region — confirm whether other attributes may be added).
- Q3: Should order edits (e.g., PricePerUnit or Qty change) create new SCD rows for ordersummary, or should order-level changes be handled separately? Business to confirm.
- Q4: Date timezone: which timezone to use for Date field normalization? (UTC recommended).
- Q5: Retention policy and archival timeline for historical SCD rows.
- Q6: Are EmailId and other PII permitted in aggregated table customeraggregatespend? If not, consider hashing or excluding.

12. Data Model Summary (logical)

- customer (Delta)
- CustId (PK)
- Name
- EmailId
- Region
- _ingest_time, _file_name (metadata)
- order (Delta)
- OrderId (PK)
- ItemName
- PricePerUnit (DECIMAL)
- Qty (INT/DECIMAL as needed)
- Date (DATE or TIMESTAMP)
- CustId (FK to customer)
- TotalAmount (DECIMAL)
- _ingest_time, _file_name (metadata)
- ordersummary (SCD Type 2 Delta)
- CustId
- Name
- EmailId

- Region
- OrderId
- ItemName
- PricePerUnit
- Qty
- Date
- TotalAmount
- StartDate
- EndDate
- IsActive
- RecordHash
- SourceMetadata
- customeraggregatespend (Delta)
- Name
- Date
- TotalAmount

13. Implementation Roadmap (high-level)

- Week 0–1: Requirements sign-off, access approvals, environment setup.
- Week 1–2: DLT pipeline design and small POC ingest of CSV -> bronze -> clean.
- Week 2–3: Implement cleansing, TotalAmount calculation, dedupe, and tests.
- Week 3–4: Implement SCD Type 2 logic (ordersummary) and test cases.
- Week 4–5: Implement aggregation to customeraggregatespend and performance tuning.
- Week 5–6: User acceptance testing, security review, documentation, and production deployment.

14. Acceptance Criteria

- Source CSV files are ingested into Delta customer and order tables at configured cadence.
- TotalAmount computed correctly for all order records.
- No records with null/ "Null" in required fields exist in production customer and order tables; rejected records visible in quarantine.
- Duplicate records removed per business rules; dedupe logs are available.
- ordersummary exists as SCD Type 2 table; when customer attributes change, historical records have EndDate and IsActive = false, and new rows are inserted with StartDate and IsActive = true.
- customeraggregatespend contains correct daily aggregated TotalAmount by Name and Date.
- All processing implemented with Delta Live Tables, observable in DLT UI with expectations and metrics.

15. Appendix

- Source file paths:
- customer: /Volumes/gen_ai_poc_databrickscoe/sdlc_wizard/customerdata
- order: /Volumes/gen_ai_poc_databrickscoe/sdlc_wizard/orderdata
- Target catalog/schema:
- gen_ai_poc_databrickscoe.sdlc_wizard
- Provided source schemas:
- customer: CustId, Name, EmailId, Region
- order: OrderId, ItemName, PricePerUnit, Qty, Date, CustId
- Aggregation:
- GROUP BY Name, Date
- SUM(TotalAmount) AS TotalAmount
- Implementation technology:
- Delta Lake, Delta Live Tables, Databricks Delta MERGE

If you approve this BRD, next steps are: confirm open decisions (Section 11), finalize retention and SLA values, and begin environment provisioning and DLT pipeline development.

Data Flow Diagram (DFD)

